

Informe de Avance: Estandarización y Preprocesamiento de Benchmarks TSPLIB para Algoritmos Evolutivos

Kevin Ariel Ramirez Amaya

16 de marzo de 2026

Resumen

El presente documento detalla los avances correspondientes a la fase inicial del proyecto de investigación enfocado en la evaluación de algoritmos evolutivos guiados por redes de Erdős-Rényi. Se expone la justificación técnica del desarrollo de un pipeline de preprocesamiento para adaptar las instancias clásicas del Traveling Salesman Problem (TSPLIB) a la arquitectura de software del modelo propuesto por Bucheli et al. (2024), así como la resolución de cuellos de botella computacionales detectados en el manejo de estructuras de datos masivas.

1. Contexto y Justificación del Trabajo Desarrollado

El código base original del repositorio asociado al artículo *“Evolutionary Algorithms Guided by Erdős-Rényi Complex Networks”* fue diseñado con fines demostrativos, inicializando el espacio de búsqueda mediante la generación aleatoria de coordenadas para las ciudades. Para cumplir con el objetivo de evaluar empíricamente el algoritmo sobre un estándar riguroso, fue imperativo desarrollar un módulo intermedio (parser/preprocesador) que actúe como puente entre los archivos crudos del estándar internacional TSPLIB y las estructuras de datos que el algoritmo evolutivo es capaz de consumir.

2. Arquitectura Detallada del Pipeline de Preprocesamiento

La adaptación de las instancias de TSPLIB al entorno del algoritmo genético propuesto por Bucheli et al. requirió el diseño e implementación de un pipeline de datos (*Data Pipeline*) compuesto por cuatro etapas secuenciales. Este enfoque modular garantiza que la preparación de los datos sea estrictamente independiente del motor evolutivo, aislando posibles fallos y facilitando la reproducibilidad.

2.1. Etapa 1: Parsing

Los archivos del estándar TSPLIB (formato `.tsp`) estructuran la información mezclando metadatos con coordenadas espaciales. Se desarrolló un analizador léxico (*parser*)

secuencial que omite las cabeceras descriptivas (como `NAME`, `TYPE` y `DIMENSION`) e identifica dinámicamente el bloque de datos empíricos mediante la directiva `NODE_COORD_SECTION`. A partir de este punto, el algoritmo extrae las tuplas de coordenadas cartesianas (X, Y) , aplicando rutinas de sanitización para manejar delimitadores irregulares y saltos de línea, deteniendo la lectura de forma segura al detectar la bandera `EOF`.

2.2. Etapa 2: Validación Topológica Bidimensional

Para mitigar la propagación de “errores silenciosos” (por ejemplo, el desplazamiento de coordenadas durante la lectura), el pipeline incluye una etapa de verificación empírica. Utilizando la librería `Matplotlib`, el sistema proyecta las coordenadas extraídas en un plano cartesiano 2D, generando un artefacto visual (`.png`). Esta salida permite al investigador confirmar visualmente que la morfología de la red de ciudades original se ha preservado íntegramente antes de iniciar el gasto computacional del modelo evolutivo.

2.3. Etapa 3: Transformación Métrica y Grafo de Costos

El motor del algoritmo evolutivo y la red de Erdős-Rényi no operan sobre el espacio euclidiano directamente, sino sobre los costos de las aristas del grafo. Por lo tanto, el pipeline transforma las coordenadas espaciales continuas en un grafo completo ponderado, representado matemáticamente como una matriz cuadrada de distancias de tamaño $N \times N$.

Cumpliendo estrictamente con el formato `EUC_2D` definido por la literatura oficial de TSPLIB (Reinelt, 1991), la distancia de costo entre cualquier nodo i y un nodo j se calcula aplicando la norma euclidiana redondeada al número entero más cercano:

$$Cost(i, j) = \left\lceil \sqrt{(X_i - X_j)^2 + (Y_i - Y_j)^2} \right\rceil \quad (1)$$

Esta matriz condensa toda la topología del problema en un formato de consulta instantánea ($O(1)$) para la función de aptitud (*fitness*) del algoritmo genético.

2.4. Etapa 4: Serialización y Puente de Software (Bridging)

La etapa final del flujo de trabajo resuelve el acoplamiento (*bridging*) entre el preprocesador y el entorno principal de ejecución (*Jupyter Notebook*). El pipeline fue diseñado para emitir los tensores resultantes en dos modalidades estratégicas:

- **Exportación Tabular (`.txt`):** Generación de una matriz plana y legible por humanos, orientada a la auditoría manual y la depuración (*debugging*) de las primeras iteraciones de prueba.
- **Serialización Binaria (`.npy`):** Formato definitivo de empaquetado profundo basado en la librería `NumPy`. Este formato comprime la matriz y permite su inyección posterior en la memoria RAM del algoritmo evolutivo esquivando los cuellos de botella de Entrada/Salida (I/O).

3. Dificultades Técnicas: El Cuello de Botella Espacial y su Optimización

Durante las primeras pruebas de desarrollo del preprocesador se identificó una dificultad crítica relacionada con la escalabilidad y el costo computacional (alineado con el Objetivo 6 del plan de estudio).

Inicialmente, el cálculo de la matriz de distancias euclidianas se implementó utilizando bucles iterativos nativos de Python ($O(N^2)$) y se optó por serializar los resultados en archivos de texto plano (.txt). Sin embargo, este enfoque demostró ser computacionalmente inaceptable:

- **Latencia de Procesamiento:** Para instancias masivas de TSPLIB, el cálculo iterativo secuencial tomaba tiempos prohibitivos (horas) para procesar una sola matriz.
- **Desbordamiento de Almacenamiento:** El guardado tabular en formato .txt generaba archivos de pesos excesivos, creando un cuello de botella grave de Entrada/Salida (I/O) al intentar inyectar estos datos en el algoritmo genético posteriormente.

Solución Implementada:

Para mitigar esta dificultad técnica, se descartó el enfoque nativo y se refactorizó el pipeline utilizando *vectorización matemática* de bajo nivel. Mediante la librería SciPy (`scipy.spatial.distance.cdist`), el cálculo de la distancia euclidiana se delegó a lenguaje C subyacente, reduciendo el tiempo de procesamiento de horas a escasos milisegundos. Además, se reemplazó la salida de texto por el formato de compresión binaria .npy (NumPy), minimizando drásticamente la latencia de carga en memoria para los futuros experimentos.

4. Próximos Pasos en la Investigación

Superado el desafío del tratamiento masivo de datos, las siguientes fases son:

- **Inyección de la Matriz:** Modificar el *Jupyter Notebook* original para que consuma los archivos .npy en lugar de instanciar grafos aleatorios.
- **Pruebas Preliminares (Smoke Tests):** Ejecutar el algoritmo guiado por Erdős-Rényi con una instancia preparada para verificar la convergencia.
- **Diseño Experimental Definitivo:** Establecer los barridos de parámetros (probabilidades de mutación y p de Erdős-Rényi) para la ejecución a gran escala.

5. Reflexión y Estado Actual del Proyecto

Durante estas primeras seis semanas, el progreso en la recolección de métricas finales ha sido menor a lo proyectado. Esto se debe a la pronunciada curva de aprendizaje que implicó auditar la arquitectura del código base y, crucialmente, enfrentarse a los problemas de complejidad espacial y temporal detallados en la Sección 3.

Si bien reconozco y me disculpo por la dilación frente al cronograma inicial respecto a la obtención de resultados finales, este periodo de control de daños”, refactorización y optimización era un obstáculo de ingeniería ineludible. Considero que el haber detectado y solucionado la ineficiencia del $O(N^2)$ desde el preprocesamiento ha dejado cimientos robustos: la ejecución del algoritmo genético en las semanas venideras se desarrollará de manera fluida, rápida y escalable.

Referencias

- Repositorio Github: <https://github.com/KevinRamirezAmaya/erdos-reny-parser.git>